Lecture 1.5: Conditionals, Parsing, and Validation

Goal of this lecture

Your rough lecture builds a small example:

- Ask for an amount (input is text)
- Convert it to a number
- Validate it (check if it's a real number)
- Apply discount based on VIP confirmation

Input from `prompt` is always text

Even if the user types `500`, the returned value is `"500"` (a string). To do math, you must convert to a number.

Converting text to number

- `parseFloat("12.5")` → `12.5`
- `parseInt("12.5")` → `12` (drops decimals)
- `Number("12.5")` → `12.5`

Validation using `isNaN`

If parsing fails, JavaScript returns `NaN` (Not-a-Number). We check it using `isNaN(value)`.

Discount example (refined and safe)

Added: handling Cancel and trimming input:

Powered by SPRK Technologies

```
var amtText = prompt("Enter amount:");

if (amtText === null) {
    console.log("User cancelled");
} else {
    amtText = amtText.trim();
    var amt = parseFloat(amtText);

    if (amtText === "" || isNaN(amt)) {
        console.error("Invalid amount");
    } else {
        var isVip = confirm("Are you a VIP customer?");

        // discount percentage
        var discPer = isVip ? 0.10 : 0.05;

        var final = amt - (amt * discPer);
        console.log(`Final amount is ${final}`);
    }
}
```

Output:

If user enters `1000` and clicks VIP OK → final is `900`.

If user enters `1000` and clicks Cancel for VIP → final is `950`.

`if / else` vs ternary

Multiple ways to set the discount percentage.

Classic if/else

```
var discPer;
if (isVip) {
    discPer = 0.10;
```


Powered by SPRK Technologies

```
} else {
    discPer = 0.05;
}
```

Ternary operator

```
var discPer = isVip ? 0.10 : 0.05;
```

> **Tip:** Use ternary for simple choices. Use `if/else` when logic is more complex or needs multiple statements.

Operators

Arithmetic

Operator	Example
`+`	`1 + 2` → `3`
`-`	`5 - 2` → `3`
`*`	`3 * 4` → `12`
`/`	`10 / 2` → `5`
`%`	`10 % 3` → `1`
`**`	`2 ** 3` → `8`

Assignment

Operator	Example
`=`	`a = 5`
`+=`	`total += 5`

Powered by SPRK Technologies

Operator	Example
`-=`	`count -= 1`
`*=`	`n *= 2`
`/=`	`sum /= 3`
`%=`	`x %= 2`
`**=`	`p **= 2`

Comparison

Operator	Example
`==`	`"5" == 5` → `true`
`===`	`"5" === 5` → `false`
`!=`	`5 != "6"` → `true`
`!==`	`5 !== 5` → `false`
`>`	`7 > 3`
`<`	`2 < 4`
`>=`	`score >= 50`
`<=`	`temp <= 100`
`in`	`"toString" in obj`
`instanceof`	`date instanceof Date`

Powered by SPRK Technologies

Logical

Operator	Example
`&&`	`loggedIn && isAdmin`
`||`	`value || defaultVal`
`!`	`!isReady`

Nullish / Logical assignment

Operator	Example
`??`	`name = input ?? "Guest"`
`&&=`	`obj.ready &&= true`
`||=`	`opts.enabled ||= true`
`??=`	`config.timeout ??= 5000`

Bitwise

Operator	Example
`&`	`a & b`
`|`	`a | b`
`^`	`a ^ b`
`~`	`~x`
`<<`	`1 << 2`
`>>`	`val >> 1`
`>>>`	`val >>> 1`

Powered by SPRK Technologies

String

Operator	Example
`+`	`"Hi, " + name`
`+=`	`greet += "!"`

Conditional / Ternary

Operator	Example
`?:`	`status = ok ? "PASS" : "FAIL"`

Unary

Operator	Example
`+`	`+"42"` → `42`
`-`	`-x`
`!`	`!isTrue`
`~`	`~n`
`typeof`	`typeof amount`
`void`	`void 0`
`delete`	`delete obj.key`

Increment / Decrement

Operator	Example
`++`	`i++`
`--`	`i--`

Powered by SPRK Technologies

Member access / Call / Optional chaining

Operator	Example
`.`	`obj.name`
`[]`	`arr[0]`
`()`	`fn()`
`?.`	`user?.address?.city`

Comma

Operator	Example
`,`	`x = (a = 1, a + 2)`

Variable naming rules (refined)

- Do not start names with a number: `1name` is invalid.
- Do not use spaces: use camelCase instead.
- Underscore `_` is allowed.
- Avoid reserved keywords (like `if`, `for`, `function`).
- Prefer descriptive names: `amount` is better than `a`.

Camel case example

```
// Good
var studentAge = 20;
var isVipCustomer = true;

// Not recommended
var siint = 10; // unclear
```


Powered by SPRK Technologies

Practice tasks

1. Ask for marks (0–100). Validate input and print “Pass” if marks ≥ 50 else “Fail”.
2. Ask for a price and a discount percentage. Validate and compute final price.
3. Rewrite an if/else block using a ternary operator (only if it stays readable).

JS Refined Notes — Lecture 1.5

Powered by SPRK Technologies